Hashing, Open Addressing

CSE 4809: Algorithm Engineering
Lecture 04

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

May 12, 2026

Introduction to Hashing & Hash tables

Hashing

Hashing is a technique used to uniquely identify objects or data by mapping variable-size input data to fixed-size, smaller values (hashes) using mathematical formulas called hash functions.

- 👉 One of the most widely used data structure after arrays.
- 👉 Mainly supports search, insert and delete in $O(1)$ time on average which is more efficient than other popular data structures like arrays, Linked List and Self Balancing BST.
- 👉 Used for dictionaries, frequency counting, maintaining data for quick access by key, etc.
- 👉 Real World Applications: Database Indexing, Cryptography, Caches, Symbol Table and Dictionaries.

Direct Address Table I

- **Direct-Address Tables**: Ideal for small universes `U`. Key `k` is stored in slot `T[k]`, giving O(1) worst-case search.
- **The Reality**: In modern systems, `U` is massive (e.g., all 64-bit integers), while the actual set `K` being stored is small.
- **The Waste**: Allocating an array of size `|U|` is impossible or extremely wasteful when $|K| \ll |U|$
- **The Solution**: Use a Hash Table of size $|K| \approx |U|$ and a Hash Function h to map keys to slots.

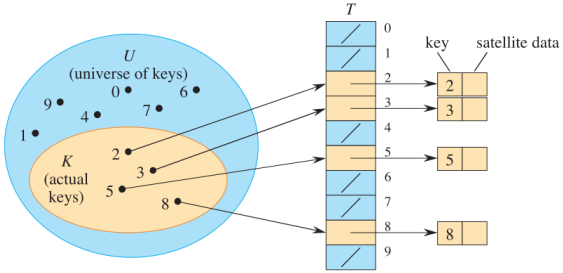

Figure: Universe `U`= {0,1,2,9 }, Set of Keys `K` = {2, 3, 5, 8}

Direct Address Table II

```
DIRECT-ADDRESS-SEARCH(T, k)
1  return T[k]

DIRECT-ADDRESS-INSERT(T, x)
1  T[x.key] = x

DIRECT-ADDRESS-DELETE(T, x)
1  T[x.key] = NIL
```

Figure: Operation Pseudocodes of Direct Address Table

Hash Tables

- **Hash Function**: Computes the slot number for the key k, so that the key goes to slot $h(k)$.
- **Hash Table**: A table $T[0 : m-1]$. The function $h(x)$ maps the universes U of keys into the slots of $T[0 : m-1]$.

$$h : U \to \{0, 1, ...m-1\}$$

where m is the size of the `hash table`. $m \ll |U|$

- **Hash Value**: Known as the slot number, the neumerical value of $h(k)$ for the key k.

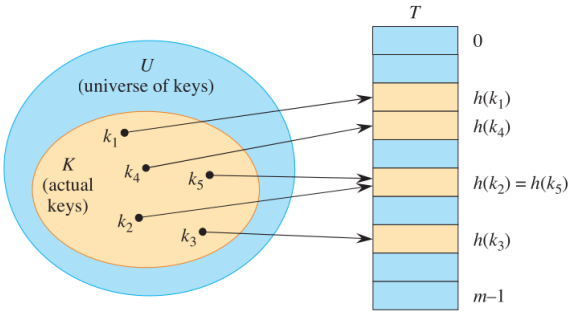

Figure: Mapping of k into hash table slots by function h

Hash Collision

Collision

Two or more keys having same hash value i.e $h(k_i) = h(k_j)$ where $i \neq j$

Independent Uniform Hashing

Ideal hash function h have output $h(k)$ where $k \in U$ such that value of $h(k)$ is an element randomly and independently chosenuniformly from the range $\{1, 2, ...m-1\}$.

- 👉 Same input k yields same hash value $h(k)$
- 👉 No Collision if $|K| \leq |T|$.
- 👉 It is and ideal abstraction. Can not reasonably be implemented in practice.

Collision Resolution by Chaining

- 👉 Each non empty slot points to a linked list.
- 👉 All keys k that has same hash value $h(k)$ goes to same slot i.e slot's linked list.
- 👉 Slot j stores all the keys k_i whose hash value $h(k_i) = j$.
- 👉 Wost case runing time:
  - Insertion: $O(1)$
  - Search: $O(\text{length of linked list})$
  - Delete: $O(1)$

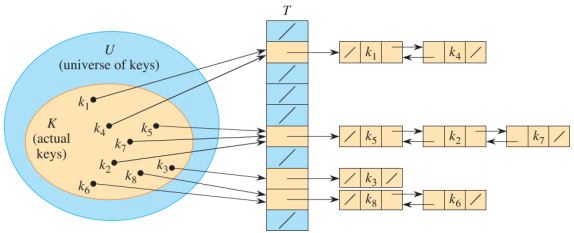

Figure: Collision Resolution by Separate Chaining

Analysis of Chaining I

Load Factor

Given a hash table T with m slots that stores n elements, `Load Factor` $\alpha = n/m$.

Assumptions:

- 👉 Given element key is equally likely to be hashed into any of the m slots i.e h is *uniform*
- 👉 Where given element hashes to, is independent of where any other elements hash to i.e *Independent Uniform Hashing*.

Analysis:

- 👉 The chance that any two distinct keys k_1 and k_2 collide is at most $1/m$. Why? [Follow the board].
- 👉 Denote length of $T[j]$ by n_j. then $n = n_0 + n_1 + n_2 + ...n_{m-1}$.
  Expected value of n_j i.e $E[n_j] = \alpha = n/m$. Why? [Follow the board].

Analysis of Chaining II

Theorem 11.1

In a hash table in which collisions are resolved by chaining, an unsuccessful search takes $\Theta(1+\alpha)$ time on average, under the assumption of independent uniform hashing.

Proof

The expected time to search unsuccessfully for a key k is the expected time to search to the end of list $T[h(k)]$, which has expected length $E[n_{h(k)}] = \alpha$
The cost of calculating $h(k)$ is $\Theta(1)$.
Total cost of unsuccessful search is $\Theta(1+\alpha)$

Theorem 11.2: Successful Search with Chaining

Theorem Statement:
In a hash table in which collisions are resolved by chaining, a successful search takes $\Theta(1+\alpha)$ time on average, under the assumption of independent uniform hashing.

Core Assumptions:

- 👉 The element being searched for is equally likely to be any of the n elements stored in the table.
- 👉 **Independent Uniform Hashing:** Each key is equally likely to hash to any of the m slots, independently of other keys.
- 👉 **Head-Insertion:** New elements are placed at the front of the list.

Step 1: Defining Search Cost

Let x be the element we are searching for. The number of elements examined during a successful search is:

$$\text{Cost} = 1 + (\text{Elements appearing before } x \text{ in the list})$$

The Head-Insertion Property:
Because new elements are inserted at the front of the list, any element appearing before x in its linked list must have been inserted into the table *after* x was inserted.

Step 2: Indicator Random Variables

Let x_i be the i-th element inserted into the table, for $i = 1, 2, \ldots, n$. Let $k_i = x_i.key$.
For each slot q and distinct keys k_i, k_j, we define the indicator variable:

$$X_{ijq} = I\{\text{search is for } x_i, h(k_i) = q, \text{ and } h(k_j) = q\}$$

Under independent uniform hashing:

- 👉 $Pr\{\text{search is for } x_i\} = 1/n$
- 👉 $Pr\{h(k_i) = q\} = 1/m$
- 👉 $Pr\{h(k_j) = q\} = 1/m$

Thus, $E[X_{ijq}] = Pr\{X_{ijq} = 1\} = \frac{1}{nm^2}$.

Step 3: Calculating Expected Cost

Let Z be the random variable counting elements appearing in the list *prior* to the target element. By linearity of expectation:

$$
\begin{aligned}
E[Z+1] &= 1 + E\left[\sum_{j=1}^{n}\sum_{q=0}^{m-1}\sum_{i=1}^{j-1} X_{ijq}\right] \\
&= 1 + \sum_{j=1}^{n}\sum_{q=0}^{m-1}\sum_{i=1}^{j-1}\frac{1}{nm^2} \\
&= 1 + \frac{1}{nm^2}\cdot m\sum_{j=1}^{n}(j-1) \\
&= 1 + \frac{1}{nm}\left(\frac{n(n-1)}{2}\right) = 1 + \frac{n-1}{2m}
\end{aligned}
$$

Step 4: Final Derivation and Conclusion

The expected number of elements examined is:

$$1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

Asymptotic Efficiency:

- 👉 As n increases, the term $\frac{\alpha}{2n}$ becomes negligible.
- 👉 The total time includes the $O(1)$ time to compute the hash function and access the slot.
- 👉 Total average time $= \Theta(2 + \frac{\alpha}{2} - \frac{\alpha}{2n}) = \Theta(1 + \alpha)$.

Conclusion: If $n = O(m)$, then $\alpha = O(1)$, and a successful search runs in **constant time** on average.

Overview: What Makes a Good Hash Function?

- 👉 **Ideal Assumption:** A good hash function satisfies **independent uniform hashing**, where each key is equally likely to hash to any slot, independently of other keys.
- 👉 **Deterministic Reality:** In practice, a hash function must be deterministic ($h(k)$ always yields the same result for input k)
- 👉 **Adversarial Risk:** For any fixed (static) hash function, an adversary can choose n keys that all hash to the same slot, forcing $\Theta(n)$ search time.
- 👉 **Engineering Constraint:** Functions must be efficiently computable (e.g., using a few machine instructions).

Static Hashing: The Division Method

Formula:

$$h(k) = k \mod m$$

- 👉 **Pros:** Extremely fast; requires only a single division/remainder operation.
- 👉 **Cons:** Performance depends heavily on the choice of m.
- 👉 **Engineering Advice:**
  - **Avoid:** Powers of 2 ($m = 2^p$), as $h(k)$ would only depend on the p lowest-order bits.
  - **Recommended:** Choose m to be a **prime number** not too close to an exact power of 2.

Static Hashing: The Multiplication Method

Formula:

$$h(k) = \lfloor m(kA \mod 1) \rfloor$$

where $0 < A < 1$.

1. Multiply key k by constant A.
2. Extract the fractional part $(kA \mod 1)$.
3. Multiply the fraction by m and take the floor.

Advantages:

- The value of m is not critical; typically we choose $m = 2^p$ for easy hardware implementation.
- Knuth suggests $A \approx (\sqrt{5} - 1)/2 \approx 0.6180339887\ldots$.

The Multiply-Shift Method

Designed for high performance where $m = 2^\ell$ and machine word size is w bits.

Formula:

$$h_a(k) = (k \cdot a \mod 2^w) \gg (w - \ell)$$

- 👉 **Step 1:** Multiply w-bit key k by a w-bit constant $a = A \cdot 2^w$.
- 👉 **Step 2:** The product ka is $2w$ bits. Taking it modulo 2^w extracts the lower w-bit half (r_0).
- 👉 **Step 3:** A logical **right shift** keeps the ℓ most significant bits of r_0.
- 👉 **Efficiency:** Executable in just **three machine instructions**: multiplication, subtraction, and shift.

Universal Hashing: Motivation

- 👉 **Random Hashing:** To defeat adversaries, we choose the hash function **randomly** from a family $\mathcal{H}$ at runtime .
- 👉 **Definition:** A family $\mathcal{H}$ is **universal** if for every pair of distinct keys k_1, k_2, the number of functions $h \in \mathcal{H}$ for which $h(k_1) = h(k_2)$ is at most $|\mathcal{H}|/m$.
- 👉 **Probability:** If h is picked uniformly at random from $\mathcal{H}$, the probability of collision is:

$$Pr\{h(k_1) = h(k_2)\} \leq 1/m$$

Hashing Long Inputs: Vectors and Strings

For inputs like DNA sequences or strings that exceed 64 bits.

- **Vector View:** Treat a string as a d-tuple $\langle a_0, a_1, \ldots, a_{d-1} \rangle$.
- **Universal Polynomial Hash:**

$$h_b(\langle a_0, \ldots, a_{d-1} \rangle) = \left(\sum_{j=0}^{d-1} a_j b^j \right) \mod p$$

where b is a random value from $\mathbb{Z}_p$.

- **Insight:** This treats the string as a number in base b and computes its value modulo p.

Introduction to Open Addressing

- 👉 **Core Concept:** Unlike chaining, all elements occupy the hash table itself. Each entry contains either an element or NIL.
- 👉 **No External Storage:** There are no linked lists or pointers stored outside the table .
- 👉 **Load Factor (α):** Since each slot holds at most one element, the number of stored keys n cannot exceed the number of slots m. Thus, $\alpha = n/m \leq 1$.
- 👉 **Algorithm Engineering Advantage:** Avoiding pointers frees up memory, allowing for a larger table size m in the same amount of space, which potentially reduces collisions.

The Probing Mechanism

- **Probing:** The process of successively examining table slots until an empty slot is found (for insertion) or the target key is located (for search).
- **Extended Hash Function:** To determine the sequence of slots to examine, the hash function includes the *probe number* (i) as a second input:

$$h : U \times \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}$$

- **The Permutation Requirement:** For every key k, the probe sequence $\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$ must be a **permutation** of $\langle 0, 1, \ldots, m-1\rangle$.

Insertion and Search Algorithms

HASH-INSERT(T, k):

$i = 0$
repeat
  $q = h(k, i)$
  if $T[q] == NIL$ **then**
    $T[q] = k$
    return q
  else
    $i = i + 1$
  end if
until $i == m$
error "hash table overflow"

HASH-SEARCH(T, k):

- 👉 Probes the same sequence as insertion.
- 👉 Terminates successfully if k is found.
- 👉 Terminates unsuccessfully if it hits a NIL slot, as k would have been inserted there.

Deletion in Open Addressing

- **The Problem:** Simply setting a slot to NIL can break the probe sequence for keys inserted later, making them unreachable.
- **The Solution: Special Sentinel:** Mark the deleted slot with a special value `DELETED` instead of NIL.
- **Algorithm Adjustments:**
  - **Search:** Must pass over `DELETED` values to continue the probe sequence.
  - **Insert:** Can treat `DELETED` as an empty slot and reuse it.
- **Engineering Note:** When many deletions occur, search times no longer depend strictly on α. Chaining is often preferred if deletions are frequent.

Linear Probing

Formula:

$$h(k,i) = (h'(k) + i) \mod m$$

where h' is an auxiliary hash function.

- 👉 **Mechanism:** If $T[h'(k)]$ is occupied, check the next slot, and the next, wrapping around the table.
- 👉 **Primary Clustering:** Linear probing suffers from long runs of occupied slots building up. This increases search time as clusters tend to grow and merge.
- 👉 **The Efficiency Paradox:** Despite clustering, linear probing is often the **method of choice** on modern hardware because successive probes are usually in the same **cache block**.

Double Hashing

Formula:

$$h(k,i) = (h_1(k) + ih_2(k)) \mod m$$

where h_1 and h_2 are auxiliary hash function.

- 👉 **Mechanism:** The initial probe is $h_1(k)$; the *step size* is determined by $h_2(k)$.
- 👉 **Requirement:** To search the entire table, $h_2(k)$ must be relatively prime to m.
  - Let m be a power of 2 and $h_2(k)$ be odd.
  - Let m be prime and $h_2(k) < m$.
- 👉 **Advantage:** Eliminates primary clustering by providing $\Theta(m^2)$ distinct probe sequences, behaving much closer to ideal uniform hashing.

Theoretical Analysis: Unsuccessful Search

Theorem 11.6:
In an open-address hash table with load factor $\alpha < 1$, the expected number of probes in an unsuccessful search is at most $1/(1-\alpha)$, assuming uniform hashing.

Intuition:

- 👉 The first probe always occurs.
- 👉 With probability $\approx \alpha$, the first slot is full, requiring a second probe.
- 👉 With probability $\approx \alpha^2$, the first two are full, etc.
- 👉 **Geometric Series:** $1 + \alpha + \alpha^2 + \cdots = \frac{1}{1-\alpha}$.

Result: If the table is 90% full ($\alpha = 0.9$), we expect 10 probes.

Theoretical Analysis: Successful Search

Theorem 11.8:
The expected number of probes in a successful search is at most:

$$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

assuming uniform hashing and that each key is searched with equal probability.

Comparison to Unsuccessful Search:

- 👉 A successful search follows the same path taken during insertion.
- 👉 If the table is 50% full ($\alpha = 0.5$):
  - Unsuccessful Search: 2 probes.
  - Successful Search: ≈ 1.387 probes.
- 👉 Successful searches are significantly more efficient than unsuccessful ones as α increases.

🤝 Reference & Q&A

Reference: Introduction to Algorithms, CLRS

- 👉 **Chapter 11**
- 👉 **Chapter 16**

Thank You!

👋 Thank you for your time and attention. 👋

🗨️ Any questions or clarifications. 💭